Course Assignment Repository

Create your own private repository that you will use for this course assignment. Make sure to give only **noroff-bed1** access to your repository BEFORE the deadline of the assignment.

There must be a README file committed to your GitHub repository and must include:

- The link to your hosted app on render.com
- A copy of the configuration of your .env file

Introduction

In this Course Assignment, you must create a Census application, allowing an Admin user to manually capture participants' details.

This application will be hosted on the Render platform, and all data needs to be stored in a mySQL database using the aiven.io platform.

There is no need to create views for any endpoints in this Course Assignment. The Front-end is optional, and it will not be tested or graded.

All solutions and endpoints will be tested through PostMan. On API endpoints, relevant and descriptive errors should be returned as JSON objects if any requested data is not retrieved successfully. All API endpoints need to return a JSON object as a response.

This application will need to save participants' data; an Admin user must be created, (specified in the Authentication section). The Admin user will authenticate themselves via Basic authentication.

For purposes of the Course Assignment, this means under Auth Type in the Authorization tab in postman, we will select basic auth and fill in the username and password. Only the authenticated Admin user can add, retrieve, and manipulate (CRUD operations) participant's data.

JSON structure

Define a JSON structure to represent participant data, employment details, and home location with the following fields.

Participant

- email: A string representing the participant's email address, which serves as a unique identifier.
- firstname: A string representing the participant's first name.
- lastname: A string representing the participant's last name.
- dob: A string representing the participant's date of birth in YYYY-MM-DD format.

Work

- companyname: A string representing the name of the company where the participant is employed.
- salary: A number representing the participant's annual salary.
- currency: A string representing the currency of the salary (e.g., "USD").

Home

- country: A string representing the country where the participant resides.
- city: A string representing the city where the participant resides.

The JSON structure should include these objects nested within a single top-level JSON object.

All properties must be provided in the POST request (made by Postman).

During implementation, it is unsafe to assume that data from requests will always be in the correct format. Therefore, in the back end of this application, you must validate that all properties have been provided in the requests body and if they are correctly formatted. Check that the DOB is a Date formatted (YYYY/MM/DD) and that the email address has the correct format.) If any validation fails, a descriptive error message would need to be returned, and that invalid data should not be added to the database.

Application and Authentication

Application

- The application must be an ExpressJS application.
- The application must use an environment (.env) file for all relevant configurations.

Authentication

- An Admin user must be created with the credentials:
  - login: admin
  - password: P4ssword
- These credentials needs to be stored in the database.
- All endpoints needs to be restricted; only the Authenticated Admin user can access the endpoints.

- **Basic Auth needs to be used.**
    - For purposes of the Course Assignment, this means under Auth Type in the Authorization tab in postman.
    - We will select basic auth and fill in the username and password.

Render.com

Create a new render.com application based on your repository. Please share your deployed app URL in the readme file of your github repository as well as in the .txt file on your moodle submission.

API Requirements

Participant list

- Create a handler for **/participants/add POST** endpoint, adding the provided participant's data to the database. Keep in mind the Participant record use case mentioned earlier.
- The request should have all data provided in the body as JSON.
- Create a handler for **/participants GET** endpoint, returning a list of all participants in a JSON object from the database.

Participant details

- Create a handler for **/participants/details GET** endpoint, returning the personal details of all participants (including first name, last name and email).
- Create a handler for **/participants/details/:email GET** endpoint, returning only the personal details of the specified participant (including first name, last name, dob).
- Create a handler for **/participants/work/:email GET** endpoint, returning only the specified participant's work details (including their company name and salary with currency).
- Create a handler for **/participants/home/:email GET** endpoint, returning only the specified participant's home details (including their country and city).
- Ensure only logged-in Admin users can access created endpoints.

Deleting / Updating the participant

- Create a handler for **/participants/:email DELETE** endpoint, which deletes the participant of the provided email from the database.
- Create a handler for **/participants/:email PUT** endpoint, which updates the participant of the provided email. The request should have the exact same format as for /participants/add POST request.
- Only requests from the authenticated Admin should be able to delete the participants from the list.